Clients Eligibility-Based Lightweight Protocol in Federated Learning: An IDS Use Case

Muhammad Asad, *Member, IEEE*, Safa Otoum, *Member, IEEE*, and Saima Shaukat

***Abstract*—Federated learning (FL) enables clients to train models locally, enhancing privacy by avoiding data centralization. Traditional FL assumes all clients have adequate resources, an often unrealistic expectation in heterogeneous networks with resource constraints like limited battery, memory, and bandwidth. These limitations can hinder performance, prolong convergence times, and lead to inaccurate models. To address these challenges, we introduce the Client Eligibility-based Lightweight Protocol (CELP), optimized for resource-constrained environments. CELP employs a sample-based pruning mechanism and a re-parameterized FedAvg algorithm, enhancing its management of resource variability. It also integrates an intrusion detection system to safeguard against malicious activities. Our results show that CELP significantly reduces communication overhead by up to 81.01% compared to FedAvg and up to 72.54% compared to FedProx and enhances system stability, achieving 93% accuracy on the MNIST dataset and 83% accuracy on CIFAR-10. These improvements demonstrate CELP's ability to deliver robust performance and efficiency in diverse FL scenarios.**

***Index Terms*—Federated learning, client eligibility, heterogeneous network, resource-constrained, intrusion detection.**

I. Introduction

THE LATEST advancements in the Internet of Things (IoT) require significant amounts of data for processing applications such as industrial 4.0, autonomous driving, and augmented reality [1]. To utilize effective deployment, those applications require powerful Machine Learning (ML) algorithms to exploit the generated data properly. However, conventional ML designs use a centrally located data server for training the data, which requires a considerable number of distributed IoT devices to upload their data to a third-party location, resulting in inefficient communication and a serious privacy concern. To overcome those communication and privacy concerns, it is essential to introduce edge-deployed, ML-based distributed algorithms such as Federated Learning (FL) [2]. In particular, FL allows its clients to train their local learning models with local data and only uploads the model parameters instead of the raw data to the centrally located server. Upon receiving those parameters, the centralized server aggregates those collected models and broadcasts the updated model to all the clients for the next training round. The process keeps repeating until the model reaches the desired convergence level [3]. Figure 1 shows an illustration of a typical FL system. Though FL provides an advantage in terms of communication efficiency and privacy preservation, traditional FL performs poorly in heterogeneous and resource-constrained networks. In heterogeneous FL, each client produces data independently, which leads to uneven data distribution, reducing the effectiveness of conventional stochastic gradient descent (SGD) based algorithms. Numerous techniques have been proposed to overcome this challenge; however, the existing techniques either suffer from slow convergence or a lack of resources [4].

Manuscript received 24 January 2024; revised 22 April 2024; accepted 5 May 2024. Date of publication 8 May 2024; date of current version 21 August 2024. This research was supported by the College of Technological Innovation, Zayed University (ZU), and the Technology Innovation Institute (TII) under grant numbers RIF-20130 & EU2205. The associate editor coordinating the review of this article and approving it for publication was G. Han. *(Corresponding author: Muhammad Asad.)*

Muhammad Asad and Safa Otoum are with the College of Technological Innovation, Zayed University, Abu Dhabi, UAE (e-mail: Muhammad.AkmalAsad@zu.ac.ae; Safa.Otoum@zu.ac.ae).

Saima Shaukat is with the School of Information Science and Technology, Department of Creative Informatics, The University of Tokyo, Tokyo 113-0033, Japan (e-mail: saima@g.ecc.u-tokyo.ac.jp).

Digital Object Identifier 10.1109/TNSM.2024.3398213

A. Motivation

The existing research on FL considered distributed learning and optimization [5], [6]. However, the critical differences between FL and traditional distributed optimization are system heterogeneity and statistical heterogeneity. FL offers a novel approach to building a combined global model by using its clients' model parameters as a learning resource [7]. The comprehensive descriptions of the FL challenges are covered by [8]. For example, heterogeneity handling can be done by utilizing on-device training, dealing with high communication costs, and considering low client participation. Federated Averaging (FedAvg), a previously developed FL technique [2], obtains a global model using the local models of the clients through an iterative process and optimization strategy. Even though the FedAvg algorithm has contributed substantially to the FL environment, several fundamental issues that might be seen in a heterogeneous FL scenario were neglected.

1) FedAvg assumes that randomly selected clients for training have uniform capabilities. However, in real-time scenarios, the participating clients may have different system configurations.
2) FedAvg drops the clients who can't complete their task within a certain period and does not allow clients to conduct partial or variable amounts of work.
3) FedAvg cannot ensure convergence if the bulk of clients is dropped or they return diverging model updates from the target.

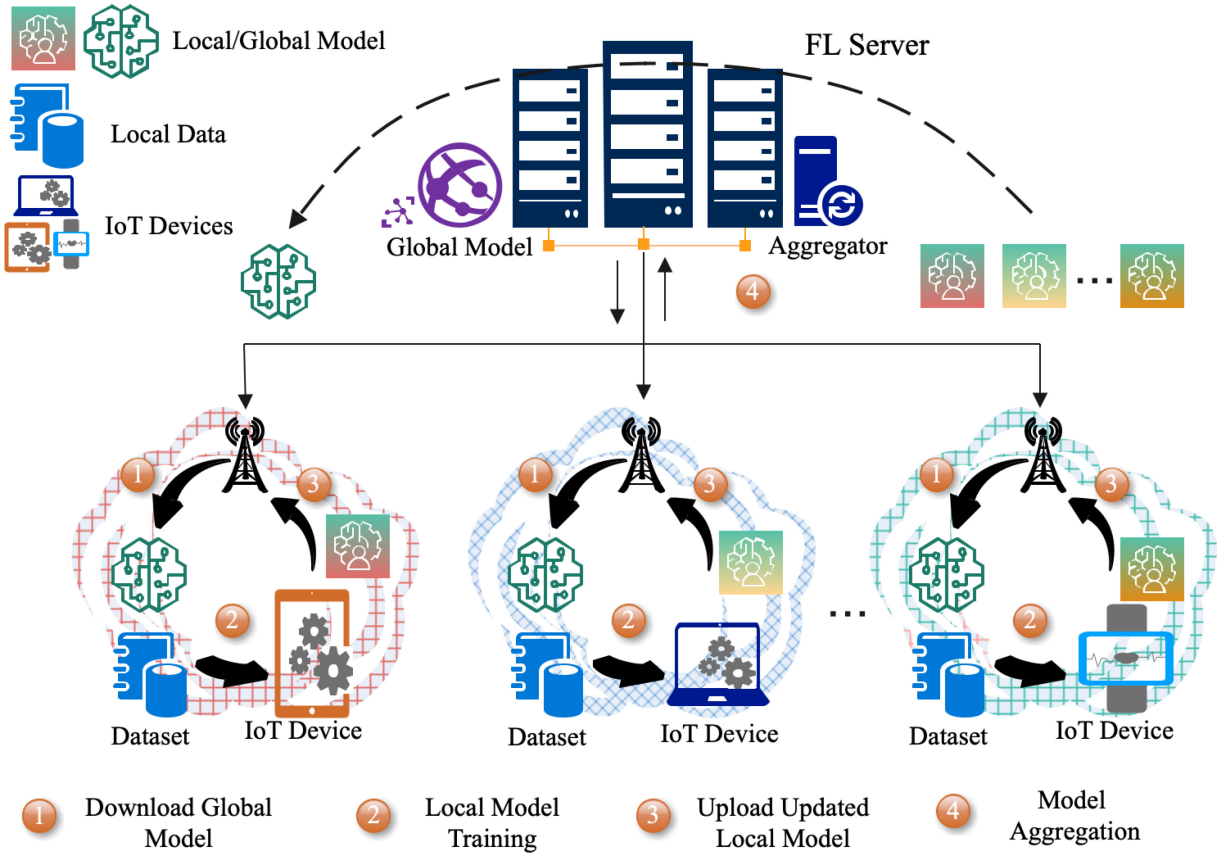

Fig. 1. The figure illustrates the typical FL system, where a set of IoT devices are connected to the cloud server. Firstly, the cloud server sends the global model to the IoT devices; secondly, those IoT devices train the local learning models individually and then upload those trained learning models to the cloud server. Finally, the cloud server aggregates those local models to generate a new global model for the next round.

4) When the client carries non-IID data, the performance of FedAvg varies significantly. This means FedAvg is not suitable for non-IID environments.

On the other hand, to mitigate these risks, our protocol integrates an advanced Intrusion Detection System (IDS) specifically designed for FL environments. This IDS actively monitors and evaluates client behavior to detect and counteract malicious activities, such as data tampering and false updates, which are critical threats in FL settings [9]. Malicious clients in a network can significantly impact the performance of FL training, as they can disrupt or manipulate the data used for training and, thus, introduce undesirable biases. In particular, malicious clients can inject false or misleading data points into the training datasets, leading to incorrect models or models that are overfitted to the malicious data. These attacks can lead to a significant decrease in the performance of FL and, ultimately, affect the performance of the trained models [10]. In this regard, our enhanced IDS within CELP not only identifies but also quantifies the extent of malicious activities, allowing for dynamic adaptation of training processes to maintain the integrity and robustness of the FL model. This implementation of IDS in CELP is crucial for maintaining high-quality, unbiased model training, especially in scenarios with a high risk of adversarial attacks. An IDS can help mitigate the effects of malicious clients in an FL network. An IDS is a system that monitors the network for malicious activities, such as DoS attacks, data manipulation, and false or modified updates. The IDSs can detect malicious clients and alert the network administrators by monitoring the network for such activities and preventing malicious clients from disrupting the training process, thus ensuring the accuracy and robustness of the models trained.

B. Our Contribution

To this end, in this paper, we propose a new FL protocol, namely, Client Eligibility-based Lightweight Protocol (CELP), that can perform efficiently in heterogeneous and resource-constrained FL environments. In particular, to reduce the model size, CELP uses a sample-based pruning technique. In addition, the CELP allows a partial amount of tasks to resource-constrained clients by measuring the clients' eligibility scores (CES). To effectively manage the challenges presented by heterogeneous networks, we have re-parameterized the state-of-the-art FedAvg algorithm. The choice of FedAvg is motivated by its proven effectiveness in aggregating model updates across diverse clients, thereby ensuring robust model convergence while mitigating the clients' variable performance and resource constraints. This adaptation further enhances CELP's capability to deliver reliable and efficient learning outcomes in diverse network conditions. The experimental results show that the proposed

protocol outperforms the existing approaches regarding stability in a highly resource-constrained environment. The significant contributions of the proposed protocol are summarized below:

- We provide a novel lightweight protocol that enables resource-constrained FL clients to train more efficiently, avoid selecting unreliable and low-resource clients (i.e., clients with limited battery life, memory, and bandwidth), and allow different local iterations depending on the client's resources.
- To reduce the model size, we utilize a sample-based pruning method that enhances the efficiency of the FL network.
- The proposed CELP is integrated with a novel incentive mechanism that selects the most reliable clients. Furthermore, the integration of IDS within CELP is not just a security feature but a strategic component that enhances overall model reliability by ensuring the integrity of client contributions. This approach is particularly beneficial in environments where client reliability varies significantly.
- The proposed CELP allows resource-constrained clients to perform partial amounts of tasks, making it more robust in real-time FL scenarios.
- We conduct extensive simulation experiments to showcase the performance of CELP in terms of pruned updates, partial participation of clients, statistical heterogeneity, and accuracy in the presence of malicious clients.

C. Paper Organization

The remainder of the paper is organized as follows: we briefly discussed the literature review in Section II. The problem statement and motivations behind the proposed CELP are defined in Section III. We presented the system models in Section IV. The complete execution process of the proposed CELP is explained in Section V. Section VI describes the convergence analysis. The simulation settings are discussed in Section VII. Experiments and Results are shown in Section VIII. Section IX presents the comparative analysis, and we conclude the paper in Section X.

II. Related Work

The latest development of the Internet of Things (IoT) requires huge amounts of data, which must reconsider how distributed optimization and learning techniques are designed [11], [12]. On one end of the spectrum, IoT services are intended for convenience in terms of architecture, speed, and Internet availability. On the other end of the spectrum, the latest developments in IoT edge devices (i.e., sensors, drones, wearables, and smartphones) allow computation at the edge devices rather than sending data to the centralized server. This results in the development of a new paradigm named FL [13]. However, this new paradigm suffers from several challenges, such as communication overhead, privacy preservation, system and statistical heterogeneity, and highly dense federated networks [14], [15]. To this end, several researchers developed novel optimization techniques (i.e., altering the direct method of multipliers (ADMM) [16] or mini-batch gradient descent [17]) that perform significantly in federated environments and outperform the conventional distributed methods. However, those techniques perform inefficiently in heterogeneous networks [18], [19]. For example, to balance the computation and communication in dense networks, the distributed optimization technique permits inexact updating of the local models that activate the limited portion of clients at every iteration. For instance, [20] proposes a multi-task learning framework to help FL clients train distinct but similar models using a primal-dual optimization technique. However, the suggested method ensures convergence but cannot be applied to non-convex problems. To overcome the non-convex problems, FedAvg involves averaging the clients' local SGD updates and outperforming the existing approaches. Moreover, to avoid the clients' statistical heterogeneity in the FedAvg algorithm, several methods considered non-federated environments, for example, [21], [22], [23] assuming the data is uniformly and identically distributed. However, it is unreasonable to presume that each local client utilizes their local data to execute the same stochastic process in the heterogeneous environment [24]. In addition, in [25], the authors developed a cooperative learning framework by considering how wireless quality, such as limited bandwidth and packet errors, affects model training. They design objective functions for the optimization problem by considering client selection, resource factors, and joint learning. Moreover, in [26], the authors studied the same issues for optimal energy consumption during transmission and model computation of FL in wireless networks. Despite the importance of those two factors (optimal energy consumption and transmission quality) in FL environments, they are not in the area of the proposed protocol.

System heterogeneity is one of the considerable challenges in the FL context; for example, different clients engaging in the training may have different amounts of battery life, memory, computing power, and bandwidth. Such variability impairs system functionality and worsens straggler problems. It may require more time or possibly fail to obtain the desired convergence if the number of stragglers increases in the network. The possible solution is to select only resourceful clients during the training process and avoid resource-constrained clients. However, stopping the straggler clients can lead to a small pool of active clients and bias in training. In addition, those clients may possess essential data in higher volume, and dropping them causes poor convergence. Beyond the heterogeneity of FL clients, model update divergence or statistical heterogeneity is also a significant challenge. Some relevant literature investigated theoretical and empirical methods to ensure convergence for FL setup [27], [28], [29]. The main issue with those FL approaches is that they assume the participating clients in training are resource-efficient and can complete the training iterations. These assumptions are false when we consider them in a realistic FL environment. To resolve statistical heterogeneity, existing works propose to share either the server's proxy data or the client's local data [30], [31]. However, disseminating the proxy information to the clients' or passing clients' information to the server may breach the privacy rules [32]. The framework proposed

TABLE I
A COMPREHENSIVE LIST OF NOTATION USED THROUGHOUT THE PAPER ALONG WITH RELEVANT ABBREVIATIONS AND THEIR MEANINGS

Notation	Description	Abbreviation	Meaning
$\mathbb{X}$	Number of clients	CELP	Client Eligibility-based Lightweight Protocol
Q	Cloud server	FL	Federated Learning
D	Local data	ML	Machine Learning
S	Size of data	IoT	Internet of Things
ω	Weight vector	SGD	Stochastic Gradient Descent
i	Particular client	TA	Trusted Authority
K_i	Impact of particular client	CES	Client Eligibility Score
G_i	Initial score for each client	FedAvg	Federated Averaging
$\mathbb{L}$	Trustworthy clients list	IDS	Intrusion Detection System
e	Training iteration	NIDS	Network-based IDS
t	Time for sending back the data	non-IID	Non-Independent and Identically Distributed
ω_r	Global model for current round	CNN	Convolution Neural Network
Ω	Model deviation		

in [7] can deal with statistical and system heterogeneity. They add a proximal term and perform a generalization of FedAvg to manage statistical heterogeneity and allow partial amounts of tasks. However, the framework is unsuitable for realistic networks as they select the clients randomly, similar to FedAvg, where most clients might be out-of-resources or inactive. Moreover, the random selection of clients may result in the selection of all straggler clients that could hardly perform a single iteration. Recently, in a rapidly evolving field of federated class-incremental learning has become increasingly crucial [33]. In federated class-incremental learning, the challenge lies in efficiently integrating new classes into the learning model without disrupting the existing knowledge structure [34].

In addition, the random selection of clients also creates a possibility of malicious clients in the network that can have detrimental effects on FL training [35]. For instance, apart from directly injecting false data, these malicious clients might engage in sophisticated network behaviors that indirectly undermine the model's integrity. Such activities could include generating anomalous network traffic to disguise data poisoning or to interfere with the communication between honest clients and the server [36]. This nuanced relationship between direct data manipulation and indirect network-level disruptions amplifies the risks in FL environments, particularly when dealing with non-IID data distributions [37]. Malicious clients, through these combined strategies, can significantly distort the training process, leading to a model that is not only poorly trained but also vulnerable to more covert attacks [38], [39]. Moreover, this dual-threat approach, direct data poisoning coupled with malicious network traffic, can also hinder the participation of legitimate clients, further exacerbating the challenges in achieving accurate and robust model training [40].

To this end, in this paper, we propose a lightweight FL protocol named CELP that can perform efficiently in heterogeneous FL environments. In particular, we divide the whole protocol into three phases. In the first phase, we perform sample-based model pruning so that clients and a server can handle the reduced model size. In the second phase, we calculate the clients' eligibility scores (CES) to select the most resourceful clients (regarding battery life, memory, and bandwidth) for training. Finally, in the third phase, we perform a re-parameterization on the FedAvg algorithm and generalize it to allow partial amounts of tasks according to the client's resources. The CELP protocol, with its focus on efficient client selection and adaptability in heterogeneous environments, shows potential for tackling federated class-incremental learning challenges. By dynamically adjusting to varying data distributions and resource constraints, CELP could effectively manage the incremental introduction of new classes, ensuring sustained learning performance without significant loss in accuracy or increase in computational load. To capture the malicious behavior of clients, we create a network-based IDS (NIDS) in CELP in the experiments to provide an extra layer of security. NIDS examines incoming traffic for suspicious patterns and behaviors and then alerts the Trusted Authority (TA) when malicious activity is detected. This allows for quick identification and remediation of any threats, ensuring the security of the network and its connected clients. The experimental results prove that the proposed CELP performs significantly in terms of convergence, especially in resource-constrained networks. For a quick reference, in Table I, we present a list of notations and terminologies used throughout the paper.

III. PROBLEM STATEMENT

In a realistic FL scenario, the clients possess heterogeneous capabilities to execute the assigned tasks. During the training process, there is a high possibility that a client may upload an inappropriate model, or experience a straggler effect. Furthermore, these unreliable and inconsistent clients can delay the training process and negatively impact the model's accuracy [41]. This scenario often arises when the server, unaware of a client's resource limitations, assigns tasks beyond their capabilities, or when the model size is too substantial for a client to handle efficiently [42].

In addition to the direct risks posed by malicious clients in FL training, such as data manipulation and communication disruptions, there is also a need to recognize and address subtler, network-level threats that these clients can introduce. Malicious clients can employ tactics that extend beyond simple data breaches or corruption; they can orchestrate complex network behaviors to create vulnerabilities in the training process [43]. This could involve generating deceptive network traffic patterns to obscure their malicious activities or interfere with the model updates between honest clients and the server.

Examples of such network-level threats include Sybil attacks, where a malicious client impersonates multiple clients to influence the training process disproportionately or replay attacks, where old network communications are reused to create confusion or inconsistencies in model updates [44].

The convergence of these direct and indirect threats significantly jeopardizes the security and integrity of the FL training process. Hence, developing robust security measures that encompass both data-level and network-level protection becomes imperative to safeguard the FL training process against such multifaceted attacks. The integrated IDS in our protocol is designed to counteract these threats by providing comprehensive monitoring and response mechanisms that address both aspects:

- Data-level Protection: Our IDS specifically looks for anomalies in data submissions, such as sudden shifts in data distribution or patterns indicative of data tampering and poisoning. We consider label flipping and clean label attacks where labels are manipulated either overtly or subtly to degrade the model's performance.
- Network-level Protection: The IDS also monitors network traffic for unusual patterns that could indicate a threat, such as an unusually high frequency of updates from a particular client or duplicate data packets that could signify a replay attack.

By integrating the IDS directly into the training and aggregation processes, we ensure that it acts not just as a passive observer but as an active defender of the network's integrity and the quality of the federated model.

IV. System Model

This section presents the system models, which serve as a foundation of the proposed Client Eligibility-based Lightweight Protocol (CELP). In particular, we first define the FL model based on the state-of-the-art FedAvg algorithm. Secondly, we present the sample-based pruning model that aims to reduce the model size. Thirdly, we explain the resource-aware client selection model that selects the most reliable and efficient clients for training. Finally, the integrated IDS is detailed as it plays a crucial role in securing the training process by monitoring and responding to data-level and network-level threats. Lastly, we present a generalization model that permits users to complete only a part of the task, depending on the remaining resources.

A. FL Model

In the proposed FL model, we assume a TA between the clients $\mathbb{X} = \{1, 2, 3, \ldots, X\}$ and a cloud server Q. To participate in the FL training, all the available clients $x \in X$ use their local data D of size S_n. The local training data consists of a pair of input and output features, where the input is the vector of data samples, and the output is the label of each input vector. The target is to minimize the loss function described as follows:

$$\min_{\omega}(F_\omega) = \sum_{i=1}^{X} K_i F_i(\omega) \tag{1}$$

where ω represents the weight vector, F_w represents the objective function, X represents the number of clients, i represents a particular client, and $K_i \geq 0, \sum_i K_i = 1$ shows each client's impact on FL model which must satisfy $\sum_i K_i = 1$. We assume that all the data samples d are available at each client and $d = \sum_i d_i$ is the total number of data points; hence, $K_i = \frac{d_i}{d}$. The cloud server disseminates a global model to the TA. Afterward, the TA announces an FL task by stating the minimum resource requirements and the CES to participate in the training. During this process, the IDS actively monitors the data exchanges for signs of data tampering or any anomalous behaviors that suggest a security threat, thereby ensuring that only genuine and secure updates are considered for model aggregation. The selected clients then train their local models for e iterations using SGD optimization. The trained models are then transmitted to the cloud server through TA. Here, IDS also checks for inconsistencies or signs of attack in the model updates received. Finally, the cloud server aggregates and obtains a new global model for the next iteration. This process keeps repeating until the convergence level is achieved. Throughout the process, we incorporate redundancy mechanisms within the data aggregation phase to counteract potential data loss or corruption. These mechanisms are designed to detect inconsistencies or anomalies in data submissions before they affect the model's accuracy. By employing a combination of real-time monitoring and post-process validation, CELP ensures that all client contributions are both authentic and constructive toward the FL objectives, thereby maintaining the integrity and reliability of the collective learning process.

B. Sample-Based Pruning Model

In a resource-constrained FL environment, the clients may have limited battery life, memory, or bandwidth, posing challenges in training large-scale ML models [45]. Our proposed CELP leverages a model-based pruning mechanism to reduce these large-scale models to smaller sizes, thus easing the computational load on participating clients [46], [47]. This pruning mechanism, applied by the cloud server, involves initially gathering data samples independently or requesting a small subset from clients. The server then uses these samples to train a neural network, combining CNN and back-propagation to learn correlations between pixels and target outputs [48]. After training, the model is pruned by removing weights and neurons that do not affect accuracy. The process of sample-based pruning in CELP is outlined as follows:

1) The server either independently gathers data samples on its own or requests a small subset of data samples from the participating clients from their shared data and assumes each client provides the data samples.
2) Initially, the cloud server initializes the global model for the first iteration. In subsequent iterations, the global model will be updated using local updates received from the clients.
3) To achieve a targeted pruning level, the server performs sample-based pruning.

4) The achieved pruned model is broadcasted to all participating clients, where each client trains their local models using the pruned global model and their local data.
5) The clients with limited resources are allowed to share partial amounts of tasks, where the cloud server aggregates the shared models of clients.

The FL process begins once the desired pruned level is achieved.

C. Resource-Aware Clients' Selection Model

In FL networks, the prevalent assumption is that all clients selected at random possess adequate resources to complete given tasks. Contrary to this, in real-world FL scenarios, client resources are often heterogeneous, potentially leading to situations where not all clients can complete their tasks. Such heterogeneity can result in stragglers, incorrect model submissions, or delayed responses, hindering the convergence of the learning process. It is, therefore, imperative that a protocol assesses clients' resources and historical performance before task assignment.

The CELP integrates a resource-aware client selection model, where the TA specifies task requirements, including minimum resource requirements such as bandwidth, memory, and battery life, alongside a Client Eligibility Score (CES) necessary for training participation. Upon gathering resource availability data from interested clients, the TA filters and selects clients who meet the training criteria, while excluding those who do not.

CES Calculation

The protocol computes the CES by identifying clients capable of executing the complete training process based on available resources. Initially, each client entering the FL network is assigned a base score of $G_i = +75$, which also serves as the eligibility threshold for training participation. A trust list $\mathbb{L}$ acknowledges clients with adequate resources and an interest in training who are not selected for the current round, granting them a motivation score of $G_t = +5$.

During training, clients who fulfill their tasks are rewarded with $G_a = +5$, and those who do not are penalized with $G_a = -5$. Timely submission of trained models is incentivized with a score of $G_m = +10$, whereas failure to submit within the deadline results in a deduction of $G_m = -5$. The protocol also maintains a historical record for each participant, imposing a penalty of $G_h = -20$ on clients failing to respond in five consecutive rounds. Furthermore, to mitigate the straggler effect, clients whose model deviates significantly from the norm are prevented from joining the training by assigning a penalty score of $G_s = -50$. These rewards and penalties are systematically added to or subtracted from the total CES. The methodology for calculating the CES across ten local iterations for three clients is illustrated in Figure 2, and the complete mechanism for client selection and CES calculation is encapsulated in Algorithm 1.

In particular, in Algorithm 1, the TA passes the training iteration (e), client id (i), given time (t) for sending back the local model, global model parameters for the current round

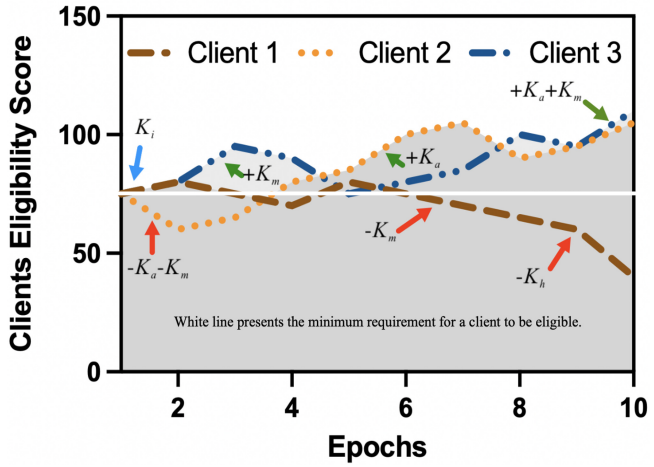

Fig. 2. The overview of clients' eligibility score for the training of three different clients, where the score is added or negated based on their performance.

Algorithm 1: Mechanism of resource-aware clients selection.

Input : current round r, local iteration (e), client id (i), global model parameters (ω_r), and model deviation (Ω);
Output: eligible clients for training

Client Eligibility Score (CES):
if *i upload local model* ω^r *within given t* **then**
  add R_i^e = 0;
  append $CES_i = G_i + G_m$;
else
  add R_i^e = +1;
  if $\frac{1}{e}\sum_i^m < r_5$ **then**
    append $H_i^e = G_m$ and $CES_i = G_i - G_m$;
  else
    if $\frac{1}{e}\sum_i^m \geq r_5$ append $H_i^e = G_h$ and $CES_i = G_i - G_h$;
  else if $R_\omega - L_i^e > \Omega$ **then**
    append $CES_i = G_i - G_s$;
Add i to trustworthy clients list $\mathbb{T}$;
Check Resource $M_i = f(\nu, \xi, \varrho)$;
if $M_i > M_{req}$;
return M_i to eligible clients list $\mathbb{EC}$;

(ω_r), and the model deviation (Ω). Each client is bound to return the local model parameters within a given time t, which the TA sets. Once the TA passes those training parameters, the CELP checks whether the client i uploaded its local model ω^r within a given time (t) or not (**Line 2**). If the client has uploaded the model within a given time, then the TA sets a record R_i^e of client i in current iteration e as 0 and rewards that client $G_m = +10$ which is added to its G_i initial score (**Line 3-4**). In contrast, the TA sets a record R_i^e of client i in current iteration e as $+1$ and checks the historical record of that client. If the sum of unsuccessful uploads is less than five consecutive rounds r, then the TA punishes the client with

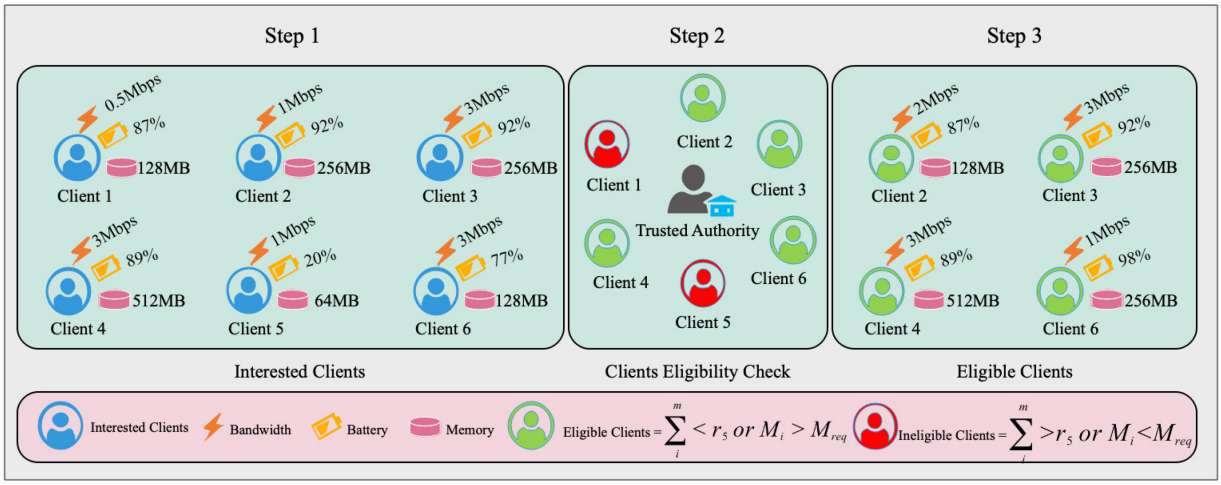

Fig. 3. The figure presents the three steps process of resource-aware clients' selection model.

$G_m = -5$ and updates it in the historical list H_i^e, which is negated from the CES (**Line 6-8**). If the historical record of that client is more than or equal to five consecutive rounds, then the TA assigns a penalty of $G_h = -20$ to that client and updates the CES accordingly (**Line 10**). If a client's model has a huge deviation from other clients' models due to the straggling effect, then the TA assigns a huge penalty to that client $G_s = -50$ so that the client won't be able to participate in the training because the client will not be able to satisfy the minimum participation eligibility score (**Line 11-12**). Once the client is added to the trustworthy list, the TA checks the client's resources (bandwidth ν, memory ξ, and battery ϱ). If the client's resources are greater than the required resources for the particular task, then the client will be added to the list of eligible clients $\mathbb{EC}$ (**Line 13-16**). The completed process of the proposed CELP is shown in Figure 3.

Algorithm 2: IDS Integrated Security Checks in FL

Input : Model updates $\{\omega_{i,t}\}_{i=1}^{X}$, Network logs $\{\text{NetLog}_{i,t}\}_{i=1}^{X}$
Output: Validated model updates, Security alerts

```
foreach client i ∈ {1, 2, ..., X} do
    ModelDeviation_i ← ||ω_{i,t} − ω_prev||;
    NetAnomaly_i ← AnalyzeNetLogs(NetLog_{i,t});
    if ModelDeviation_i < τ and NetAnomaly_i < ρ then
        Update Global Model with ω_{i,t};
    else
        Raise Security Alert for client i;
        Exclude ω_{i,t} from current aggregation;
```

D. IDS Deployment

The IDS is an integral component of our CELP, designed to ensure the integrity and security of the FL process by monitoring both data-level and network-level interactions. It dynamically detects and mitigates potential security threats, such as data poisoning and anomalous network activities, which could compromise the model's accuracy or lead to data leakage.

IDS Functionality: The IDS operates by continuously monitoring the data traffic between clients and the central server, analyzing patterns that deviate from expected behaviors. This proactive monitoring helps in the early detection of both passive and active attacks, which include but are not limited to label flipping, model poisoning, and Sybil attacks. The detection process is formalized by defining a threat model and corresponding detection criteria:

$$\text{Threat}_{i,t} = \begin{cases} 1 \text{ if } \Delta \text{ Model}_{i,t} > \tau \text{ or } \text{Network}_{i,t} > \rho \\ 0 \text{ otherwise} \end{cases} \quad (2)$$

where Δ Model$_{i,t}$ represents the deviation of the uploaded model from client i at time t from the expected model update norm, and τ is the threshold for acceptable model variation. Networki, t measures the anomalous network traffic from client i at time t, and ρ is the corresponding threshold for network anomaly detection.

IDS Implementation: The IDS is implemented through a series of checks integrated into the data aggregation and model update phases of the training process. Algorithm 2 outlines the procedure of incorporating IDS into the FL cycle, emphasizing how each step is validated for security and integrity.

The IDS ensures that only contributions adhering to established security protocols influence the global model, enhancing the overall robustness of the FL environment.

E. Generalization Model

As we previously described, the participating clients may experience heterogeneous resource restrictions regarding bandwidth, memory, and battery life. The device may lose the network connection for various reasons despite the proposed protocol's resourceful client selection using available system parameters. Moreover, there is also a possibility that all the participating clients have limited remaining resources; therefore, selecting those clients for training would be the

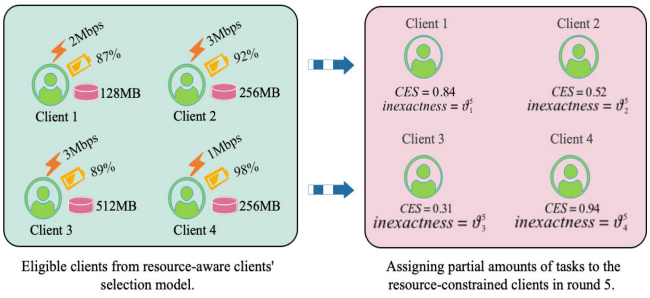

Fig. 4. The representation of four heterogeneous clients considered for partial amounts of tasks.

only choice. To this end, we considered allowing a partial number of tasks in the proposed CELP according to the client's resources.

To allow partial amounts of tasks to the resource-constrained clients, we perform generalization on the state-of-the-art FedAvg algorithm and re-parameterize it to deal with the variable performance of clients. In particular, we use the "Resource-aware Clients' Selection Model" mentioned above and select a subset of the most resourceful clients. The server performs sample-based model pruning using the collected data samples from those selected clients and reduces the model size. The reduced global model is then sent to all the participating clients. Unlike FedAvg, the proposed CELP doesn't force the clients to uniform local iterations. Instead, the TA assigns the local iterations to the participating clients, and the clients train their models to match their resource capabilities. The trained models are then sent back to the cloud server, where the aggregation is performed asynchronously. This results in maximum participation in model training, including straggler clients. Previously, in [7], the authors considered a similar scenario of partial participation of clients; however, their design doesn't perform practically in the later iterations as they had to choose the clients with minimum resources; hence, the model update deviates and results in lower accuracy. In contrast, the proposed CELP leverages the resource-aware clients' selection to tackle the straggler clients and further allow partial amounts of tasks. To consider the ϑ_i^r-inexactness for the resource-aware participating client i at the training round r, we define as follows:

Definition 1 (ϑ_i^r-inexactness): Let us consider a function $\Psi_i(\omega;\omega_r) = F_i(\omega) + \frac{\gamma}{2}\|\omega-\omega_r\|^2$, and $\vartheta \in [0,1]$, we call ω^* is a ϑ_i^r-inexactness of $\min_\omega \Psi_i(\omega;\omega_r)$ if $\|\nabla\Psi_i(\omega^*;\omega_r)\| \leq \vartheta_i^r\|\nabla\Psi_i(\omega_r;\omega_r)\|$ where $\nabla\Psi_i(\omega;\omega_r) = \nabla F_i(\omega)+\gamma(\omega-\omega_r)$.

where ϑ_i^r is a representation of variable local iteration that shows the required local computation for a particular client i in a global round r to achieve a given FL task. In particular, by relaxing the $(\boldsymbol{\vartheta_i^r - inexactness})$, we can achieve the system heterogeneity close to the optimal solution. We use this $(\boldsymbol{\vartheta_i^r - inexactness})$ solution because it allows for an approximate solution to be obtained without the need for a computationally expensive exact solution. In Figure 4, we show the overview of considered clients for partial amounts of tasks.

Algorithm 3: The Execution Process of CELP

1 Sample-based pruning: The cloud server receives a data sample from a subset of clients to perform model pruning.
2 **Server Execution:**
3 Initialize the pruned global model and send the task requirements to TA
4 New clients send their interest in joining the training;
5 TA assigns the G_i to all clients;
6 Clients receive the task requirements from the TA;
7 initialize ω_0;
8 check resources of all interested clients;
9 **for** *each round r = {1,2,3,...,n}* **do**
10 $\mathbb{EC}_r$ = ***Check Resource*** $f(\nu,\xi,\varrho)$;
11 update $\mathbb{EC}$ with updated CES;
12 **for** *each client* $i \in \mathbb{X}$ *in parallel* **do**
13 $\omega_{r+1}^i \leftarrow$ ClientUpdate (i, ω_r);
14 **if** *i updates model within given t threshold* **then**
15 **IDS Check: Perform security check on** ω_{r+1}^i **and** NetLog_i**;**
16 **if** *IDS confirms* ω_{r+1}^i *is secure* **then**
17 $\omega_{r+1} \leftarrow \omega_{r+1} + \frac{i_r}{\mathbb{D}}\omega_{r+1}^i$;
18 **else**
19 Raise Security Alert for client *i*;
20 Exclude ω_{r+1}^i from current aggregation;
21 Update CES;
22 **Client Update:** (client i);
23 Client i finds a ω_{r+1}^i which is an ϑ_i^r-inexactness of $\Psi_i(\omega;\omega_r) = F_i(\omega) + \frac{\gamma}{2}\|\omega-\omega_r\|^2$ to find the optimal number of iterations.
24 $B \leftarrow$ (split $\mathbb{H}_i$ into batches of size B);
25 **for** *each iteration e* **do**
26 **for** *batch* $b \in B$ **do**
27 $\omega \leftarrow -\eta\Delta\ell(\omega;b)$;
28 **return** ω to the cloud server;

V. Execution of CELP

We present the complete execution procedure of the proposed CELP in Algorithm 3. The execution begins with the cloud server performing sample-based model pruning, using data samples received from a subset of clients (**Line 1**). The server then initializes the pruned global model and communicates the task requirements to the TA (**Lines 3-6**). The TA assesses the clients' interest in training and assigns an initial score G_i to all clients, along with the task requirements and system parameters (**Lines 4-8**). Each round of the process checks the resources of interested clients and updates the eligible clients list based on the CES (**Lines 9-11**).

During each round of training, the protocol ensures security through an integrated IDS that performs checks on model updates and network logs. If the IDS detects a threat, it raises an alert and the affected client's update is excluded from the current aggregation (**Lines 14-20**). This security measure

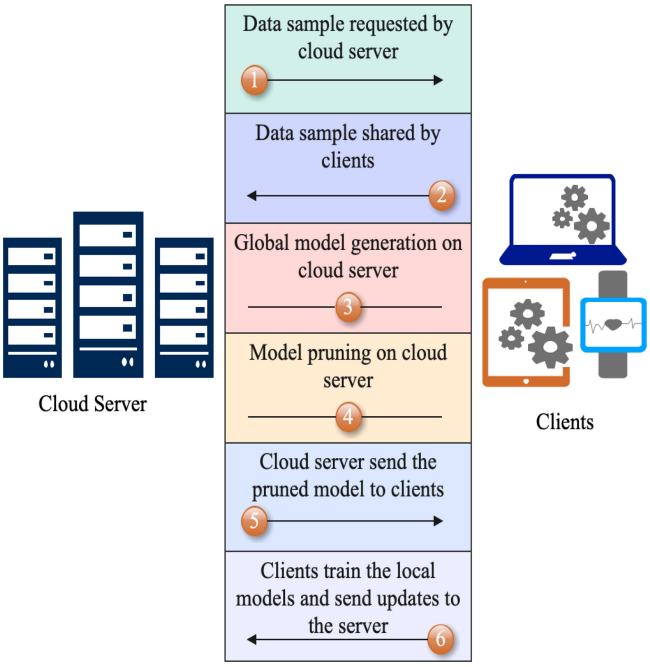

Fig. 5. The execution flow of the proposed CELP.

is crucial for maintaining the integrity of the FL process. Following security checks, clients conduct local training, split their data into batches, and optimize using SGD (**Lines 23-28**). The cloud server collects the local models' parameters from the clients and aggregates them to update the global model, ensuring that only verified and secure updates are used (**Lines 14-21**).

Finally, the TA updates the CES based on each client's response to the training requirements and their timely submission of the updated model (**Line 21**). The updated CES then informs the selection of clients for subsequent training rounds. This comprehensive approach, encapsulated in CELP, leverages asynchronous FL to accommodate varying data sizes and client capabilities, optimizing resource allocation and maintaining data security across the network. The complete flow of the proposed CELP is depicted in Figure 5.

VI. Convergence Analysis

To prove the convergence analysis, we first measure the local batch *B-dissimilarity* of the proposed CELP. In particular, *B-dissimilarity* defines a bounded dissimilarity by allowing statistical heterogeneity in independent and identical distribution (IID) scenarios.

Definition 2 (B-Dissimilarity): the local functions of the participating clients in FL training are *B-dissimilar* at ω if $\mathbb{Z}_i\ [\|\nabla F_i(\omega)\|^2] \leq \|\nabla f(\omega)\|^2 B^2$. So now we can define the value of $B(\omega)$, i.e., $B(\omega) = 1$ when $\mathbb{Z}_i\ [\|\nabla F_i(\omega)\|^2] = \|\nabla f(\omega)\|^2$. Here, all clients' local functions must agree on ω, or all clients must have the same local functions, and $B(\omega) = \sqrt{\frac{\mathbb{Z}_i[\|\nabla F_i(\omega)\|^2]}{\|\nabla f(\omega)\|^2}}$, when $\|\nabla f(\omega)\| \neq 0$.

The $\mathbb{Z}[\cdot]$ presents the expectation on FL the participating clients with masses $S_i = \frac{n_i}{n}$ and $\sum_{i=1}^{N} S_i = 1$, where n denotes the number of total data samples in the network, and n_i denotes the local data samples of a client i. Under the bounded dissimilarity, if we have $\epsilon > 0$, there is a possibility of a B_ϵ, i.e., for all the data samples in the network. However, due to the diverse data distributions across the FL environment, one can easily observe $B(\omega) > 1$, where a larger value of $B(\omega)$ denotes a greater degree of dissimilarity between the local functions of the clients.

A. Convex Scenario

We assume that $F_i(\cdot)$ is convex and $\vartheta_i^r = 0$ for any client i and round r, such that all the participating clients in the training are able to solve their local problems exactly. The other assertions are satisfied in the non-convex scenario.

B. Non-Convex Scenario

Here, we assume non-convex and L-Lipschitz smooth for the local functions F_i [49]. We assume that in the local functions there exists $L > 0$, such that $\nabla^2 F_i \geq -L_i, \bar{\gamma}{:}\gamma - L > 0$ and $B(\omega^r) \leq V$. Now, considering the formulations from Algorithm 2, we obtain

$$\lambda = \left(\frac{1}{\gamma} - \frac{\vartheta B}{\gamma} - \frac{B(1+\vartheta)\sqrt{2}}{\bar{\gamma}\sqrt{i}} - \frac{L, B(1+\vartheta)}{\bar{\gamma}\gamma} - \frac{L(1+\vartheta)^2 B^2}{2\bar{\gamma}^2} - \frac{L, B^2(1+\vartheta)^2}{\bar{\gamma}^2 i}\left(2\sqrt{2i}+2\right)\right) \tag{3}$$

Considering the equation above, we observe that in the iteration r for the local function, there is an expected loss, which is demonstrated as follows:

$$\mathbb{Z}_v\left[f\left(\omega^{r+1}\right)\right] \leq f(\omega^r) - \lambda\|\nabla f(\omega^r)\|^2 \tag{4}$$

where v denotes the subset of i devices selected in the iteration r. Similarly, we have the same assumptions for variable ϑ's, and we again consider the formulations from Algorithm 2, and we obtain

$$\lambda^r = \left(\frac{1}{\gamma} - \frac{\vartheta^r B}{\gamma} - \frac{B(1+\vartheta^r)\sqrt{2}}{\bar{\gamma}\sqrt{i}} - \frac{L, B(1+\vartheta^r)}{\bar{\gamma}\gamma} - \frac{L(1+\vartheta^r)^2 B^2}{2\bar{\gamma}^2} - \frac{L, B^2(1+\vartheta^r)^2}{\bar{\gamma}^2 i}\left(2\sqrt{2i}+2\right)\right) > 0 \tag{5}$$

Similarly, here we observe an expected loss in the global function in the iteration r, which is expressed as follows:

$$\mathbb{Z}_v\left[f\left(\omega^{r+1}\right)\right] \leq f(\omega^r) - \lambda^r\|\nabla f(\omega^r)\|^2 \tag{6}$$

where v denotes the subset of i devices selected in the iteration r and $\vartheta_r = \max_{i\in v} \vartheta_i^r$.

VII. Simulation Settings

This section shows the implementation criteria and then discusses the simulation setup and dataset representation. Finally, we show the simulation results of the proposed CELP.

A. Implementation

For the implementation, we use a sample-based model pruning that helps reduce the model size. To distribute the task, we grouped the image sample of clients as batches for input, and then we set the TensorFlow variables to construct the model. To make prediction and loss computations using those variables and model parameters. A distributed function is defined for the clients to send their local metrics to the cloud server for aggregation. After the aggregation, we use TensorFlow Federated [50] for model representation.

B. Network Configuration

In the network configuration, we build a resource-bounded edge FL environment with multiple clients that can follow the instructions. For a real-time heterogeneous environment, we set the variant configurations of memory, processor, and battery for each client. To keep the FL process simple, we consider similar transmission rates for all the clients. In particular, we consider 50 clients, assuming that twenty are unreliable, four clients have resource shortages, and four deliver low-quality models. Considering variant clients' configuration, we simulate the straggled and computation-capable clients with different resource constraints. We assess the proposed CELP framework using well-known datasets to better understand the effects of statistical heterogeneity.

C. Dataset and CNN Models

In the simulation experiments, we use two different Convolution Neural Network (CNN) models, namely, *LeNet*, and *AlexNet*. The *LeNet* is trained with the *MNIST*, a handwriting dataset. The *MNIST* dataset contains 60,000 training samples, and each sample is based on handwritten digits ranging from 0-9. The *AlexNet* is trained with the *CIFAR-10*, an image-based dataset. The *CIFAR-10* dataset contains 50,000 training samples and 10,000 testing samples, where each sample consists of images with the size of 32×32 pixels. In the case of IDS, we consider the UNSW-NB15 [51], and CICIDS2018 [52] datasets. The two datasets are popular tools for evaluating the performance of an IDS for FL networks. In particular, the UNSW-NB15 dataset is used for network security attacks and normal activities collected by the Centre for Computer Security Research of the Australian Defence Force Academy. It contains 49 features, such as IP addresses, protocol types, packet sizes, etc. This dataset provides a benchmark for testing the accuracy of anomaly detection systems for NIDS. On the other hand, the CICIDS2018 dataset is designed by the Canadian Institute for Cybersecurity to capture network traffic. It contains 81 features, such as flow duration, protocol type, source, destination IP addresses, etc. This dataset is useful for testing and validating FL algorithms for detecting and classifying cyber-attacks. For all these datasets, we employ the CNNs commonly used in the relevant literature. We present the architecture of those CNN models in Table II.

D. System Environment

We run all our experiments on the server with an Apple M1 Pro with a 10-core CPU, 16-core GPU, 16-core Neural Engine, and 64 GB of RAM. The proposed lightweight protocol is simulated by TensorFlow in Python.

TABLE II
NETWORK MODEL ARCHITECTURES

CNN	*Architecture*
LeNet [53]	C20-MP-C50-MP-FC500-FC10
AlexNet [54]	C128×3-AP16-FC10
NIDS-based	C2-MP-CF16 × 32-2×FC1-MP4

Notations: C represents the convolution layer with the given number of channels and ReLu activation. AP & MP represents the AveragePooling and MaxPooling with the given stride size. CF represents the convolution filters added to MP. FC represents the fully connected layers with the given number of neurons.

TABLE III
HYPER-PARAMETERS

Parameter	*Value*
Global communication rounds	50
Local iterations	10
Number of participants	50
Number of classes	10
Gradient Size	128 $bits$
Batch size	10
Learning rate	0.01
Local update size	10000 $nats$

E. Evaluation Schemes

To better understand the performance of the proposed CELP, we consider the state-of-art approaches FedAvg [2] and FedProx [55]. We consider the same settings while executing all three approaches. In order to highlight CELP's distinctiveness, we emphasize its historical analysis aspect in client selection, setting it apart from FedAvg and FedProx. This historical perspective in CELP, taking into account past client interactions, ensures more informed and effective client selection, enhancing the robustness and efficiency of the learning model. To bring fairness to the results, we use SGD as a local optimizer, which is also considered in both existing approaches. Moreover, we maintain the same hyper-parameters for all the approaches in all experiments; the details of those hyper-parameters are shown in Table III.

VIII. EXPERIMENTS AND RESULTS

In this section, we show the performance of the proposed CELP. In particular, we first show the performance of the pruned updates and then the performance of resource-aware clients' selection. Finally, we present convergence accuracy and the expected loss compared to the existing schemes on both *LeNet* and *AlexNet*, respectively.

A. Pruned Updates

As the central aim of the proposed protocol is to support resource-constrained FL networks, it is crucial to develop a lightweight model that prioritizes essential features to expedite training time. Our approach incorporates sample-based model pruning, leveraging TensorFlow's model-optimization

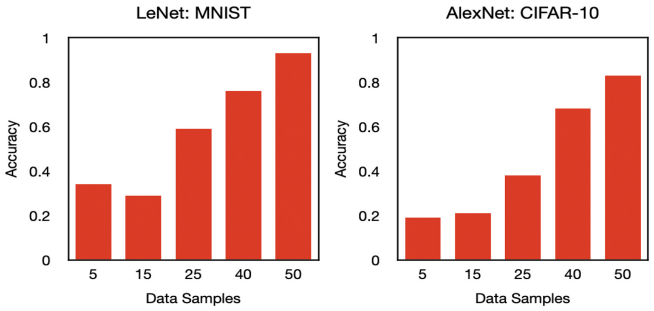

Fig. 6. Accuracy of pruned updates on variant data samples collected from the clients for training on both *LeNet* and *AlexNet*, respectively.

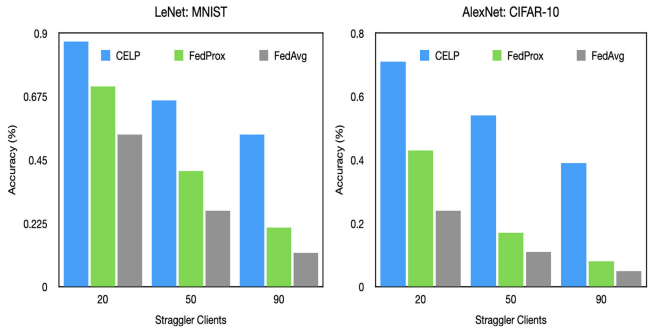

Fig. 7. Comparison of accuracy on a variant number of straggler clients for training on both *LeNet* and *AlexNet*, respectively.

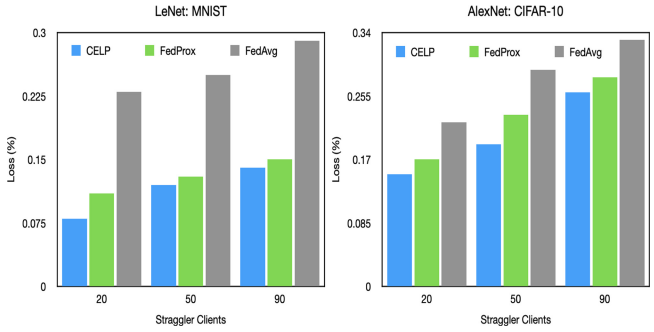

Fig. 8. Comparison of loss on a variant number of straggler clients for training on both *LeNet* and *AlexNet*, respectively.

classes to selectively prune low-magnitude classes from the Keras framework. The resulting model parameters are managed through classes like PolynomialDecay and prune low-magnitude, which facilitate efficient model updates.

To demonstrate the efficacy of our approach, we assess the performance of pruned model updates across different network architectures, including *LeNet* and *AlexNet*, as shown in Figure 6. We explore various sample sizes, i.e., 5%, 15%, 25%, 40%, and 50%, to identify the optimal balance between model performance and resource efficiency. Our findings reveal that smaller sample sizes often lead to less accurate models due to the potential exclusion of some classes. In contrast, larger sample sizes, such as 50%, significantly enhance model accuracy by ensuring a more representative sample across all classes, thereby mitigating class omission. This strategy, although seemingly counter to the typical FL paradigm of minimal data usage, is justified by the need for robust model accuracy in resource-constrained environments where each training iteration is costly. The increased sample size ensures that the model training is as comprehensive as possible within the fewest number of rounds, aligning with the ultimate goal of efficiency in FL networks.

B. Partial Participation of Clients

To allow partial amounts of tasks to the participating clients, we consider system heterogeneity to measure the performance of the proposed CELP. When the cloud server assigns the task, the TA measures the resource capabilities of each client and allows only resourceful clients to train using the resource-aware clients' selection model. However, there is a possibility that we may have only resource-constrained clients in the network; therefore, we must allow clients to perform partial tasks. To this end, a participating client i estimates the number of tasks it needs to accomplish on iteration r as a function of its available resource restrictions in a global round R. The global rounds must be performed by all the clients, and the clients who cannot perform up to the E rounds, then those clients are allowed to perform fewer iterations based on their resource capabilities. Here, i represents the participating client, r represents the local iteration, R denotes the single global round, and E denotes the complete global rounds.

In addition to system heterogeneity, the scalability of CELP is critically assessed to ensure its efficacy across varying numbers of clients. Scalability testing involves analyzing the protocol's performance as the number of participating clients increases. This is vital as the increase in client numbers could potentially exacerbate the straggler effect, impacting the overall system's convergence rates and accuracy. Our scalability investigations help identify any potential constraints, such as the increased coordination overhead and resource allocation challenges in larger networks, ensuring that CELP maintains reliable performance even as network size expands.

In Figure 7 and Figure 8, we show the performance of the straggler effect. In particular, we experimented with training accuracy and loss, respectively, on both *LeNet* and *AlexNet* by considering a variant number of straggler clients in the network. We consider three different scenarios of straggler clients, i.e., 20, 50, and 90, where 20 means that 80% of the participated clients can perform up to E rounds, and 90 means that only 10% of the participated clients can perform up to E rounds. From the results, we can observe that system heterogeneity negatively impacts the convergence performance, and a higher number of heterogeneity results in poor convergence. Moreover, it is clear that considering the straggler clients can achieve significant performance rather than dropping them in the first place. This means that allowing partial amounts of tasks to the straggling clients does not affect the overall performance. The results in Figures 6-8 indicate that sample-based model pruning may result in some accuracy loss; however, if we choose the clients efficiently, we can still obtain significant convergence while suffering from a minimal loss.

TABLE IV
COMMUNICATION BITS REQUIRED FOR UPLOAD AND DOWNLOAD TO ACHIEVE THE TARGETED ACCURACY IN THE PRESENCE OF STRAGGLER CLIENTS FOR THREE DIFFERENT SCENARIOS, I.E., 20, 50, AND 90

	LeNET: MNIST (Accuracy = 90%)	**AlexNET: CIFAR-10 (Accuracy = 85%)**
FedAvg (20)	107.4 MB/107.4 MB	2322.6 MB/2322.6 MB
FedAvg (50)	90.2 MB/90.2 MB	1836.5 MB/1836.5 MB
FedAvg (90)	54.3 MB/54.3 MB	1265.4 MB/1265.4 MB
FedProx (20)	74.3 MB/201.4 MB	1356.8 MB/3343.2 MB
FedProx (50)	61.2 MB/398.7 MB	1106.9 MB/4100.1 MB
FedProx (90)	41.1 MB/488.5 MB	749.5 MB/5464.3 MB
CELP (20)	20.4 MB/154.3 MB	179.1 MB/1476.2 MB
CELP (50)	18.6 MB/189.6 MB	201.4 MB/1603.1 MB
CELP (90)	16.1 MB/201.7 MB	211.6 MB/1821.7 MB

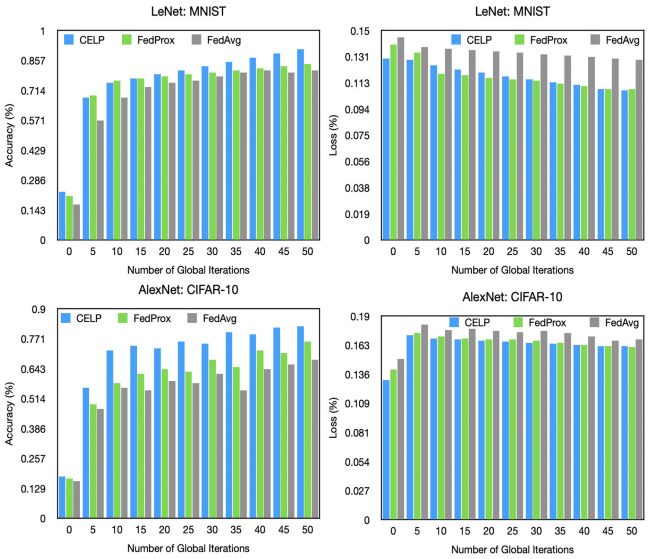

Fig. 9. Comparison of statistical heterogeneity in terms of accuracy and loss for training on both *LeNet* and *AlexNet*, respectively.

C. Statistical Heterogeneity

To check whether the proposed CELP can handle the statistical heterogeneity or not, we eliminated the proximal term from the clients' objective functions and performed a simulation experiment. By allowing heterogeneity in the dataset, the clients' performance drops significantly. In Figure 9, we show the statistical heterogeneity of the proposed CELP on the MNIST and CIFAR-10 datasets. In this experiment, we didn't consider any system heterogeneity, and we assume that all the participating clients' have enough resources to train their models up to E iterations. The authors in [2] state that tuning the number of iterations helps reach the desired convergence level. However, more iterations require more computational tasks from the clients' resulting in faster convergence but reducing the communication efficiency. Moreover, in a realistic FL network, setting up a higher number of iterations may increase the possibility that a participating client cannot perform a computational task. Therefore, it is crucial to set a sufficient number of iterations while ensuring robust convergence. In Particular, in Figure 9, we consider the same hyper-parameters as mentioned in Table II and measure the performance of the proposed CELP compared to the FedProx and FedAvg algorithms in terms of global accuracy and loss. It can be seen that the proposed CELP handles the statistical heterogeneity effectively, obtains higher accuracy, and faces lower expected loss compared to the state-of-the-art FedProx and FedAvg.

D. Communication Costs

In addition to evaluating CELP's performance in terms of accuracy and loss, we have also examined its efficiency in communication costs. This is a crucial aspect, especially in FL environments with straggler clients. We compare the communication costs required to achieve targeted accuracy levels in CELP with those in FedAvg and FedProx. Our analysis, presented in Table IV, shows the upload and download communication bits required for *LeNet* using the MNIST dataset and *AlexNet* using the CIFAR-10 dataset under different straggler scenarios (20, 50, and 90). These scenarios represent varying proportions of clients that can perform up to the complete global rounds. The results indicate that CELP significantly reduces the communication overhead compared to both FedAvg and FedProx, especially as the proportion of straggler clients increases. This reduction in communication costs is instrumental in enhancing the efficiency of FL, particularly in networks with resource-constrained clients.

E. IDS-Use Case

In the training process of FL, the malicious clients tend to launch malicious activities, and because the central server cannot access the local data, therefore, FL cannot control the poisoning attacks [56]. In such attacks, the malicious clients disturb the FL model and try to weaken the security protection of local training data by injecting poisoned traffic data. To this end, we consider two poisoning attacks, namely, label flipping and clean label attacks, in our proposed CELP-based IDS case. In particular, in a label-flipping attack, an adversary manipulates the training data labels, such as flipping benign samples to malicious labels or malicious samples to benign

labels. This can introduce noise into the training data and lead to a misclassification of test data. On the other hand, clean label attacks involve attacking a model by using carefully crafted samples that are correctly labeled with the correct labels. This may involve an adversary crafting malicious data, which is labeled as benign, or benign data, which is labeled as malicious. These samples will be fed into the model during training, potentially resulting in the misclassification of both benign and malicious data.

In the proposed experiments, we explore two different distributed data settings for FL-based NIDS: IID and Non-IID. As mentioned above, we consider two datasets, UNSW-NB15 and CICIDS2018. For the case of IID, we consider the UNSW-NB15 dataset and conduct a binary classification (anomaly detection) task because it allows all the clients to collect malicious and benign traffic. For the case of Non-IID, we consider the CICIDS2018 dataset and perform a multi-classification (intrusion detection) task that asks all clients to collect several types of traffic data while performing the local training. In the experiment, we considered 50 clients for model training; among them, 20 were malicious clients who launched poisoning attacks. We adopt the model poisoning strategy from [57] that maximizes the success of those attacks.

Attack Scenarios: We consider two attack scenarios: single and multi-round. In particular, single rounds indicate that a client injects a poisoning attack in one round, e.g., an attack is launched in a second round, then it is represented as (Λ_2). In the experiment, we consider single round attack in three different rounds Λ_5, Λ_{10}, and Λ_{20}. On the other hand, a scenario of a multi-round attack means the poisoning attack is launched in multiple rounds. In this scenario, we continuously injected poisoning attacks.

Benchmarks: The performance of CELP defensive mechanisms is compared against the state-of-the-art method, namely, VAE [58]. VAE is based on the random selection of model parameters and requires a pre-trained offline model on the server side to detect and reject anomalous local models. Its implementation is similar to that described in [58].

Evaluation: We consider the effectiveness of NIDS in the proposed lightweight protocol by launching the single-round attack scenario. As the proposed model relies on the TA, which is connected to the clients throughout the training process. Thus, we compare it with the existing offline detection-based method, VAE, for single-round attacks. Figure 10 shows the detection accuracy of both methods. It is evident that the proposed CELP outperforms VAE in both datasets, demonstrating its superior capability of detecting poisoned models.

In Figures 11 and 12, the efficacy of CELP in a NIDS context is showcased, particularly under the strain of multi-round poisoning attacks. The accuracy curves presented in Figure 11 for the UNSW-NB15 dataset indicate that CELP maintains higher accuracy compared to both FedAvg and VAE, demonstrating its resilience to label-flipping and clean label attacks. Specifically, CELP's consistent performance across multiple rounds of attacks can be attributed to the dynamic adaptation of its pruning and client selection mechanisms, which effectively mitigate the impact of poisoned updates on the global model.

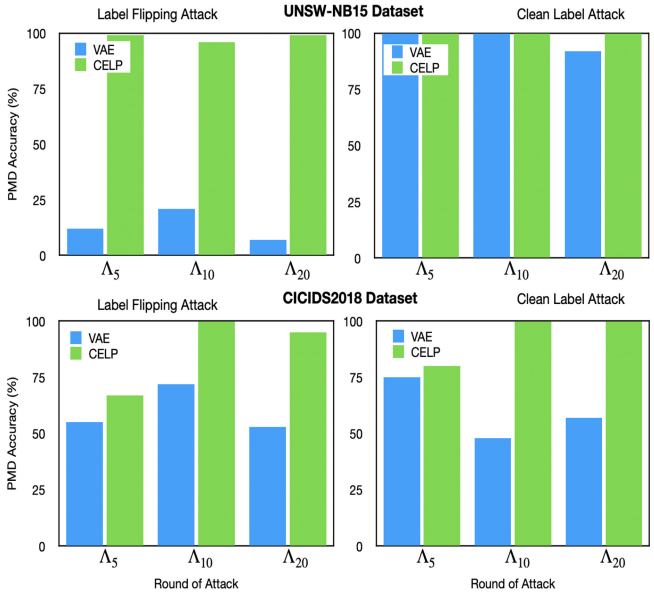

Fig. 10. Comparison of poisoned model detection (PMD) accuracy with state-of-the-art offline detection method (VAE).

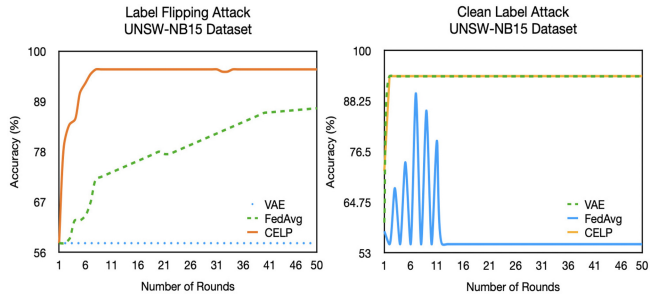

Fig. 11. Performance comparison of accuracy on multi-round attack scenario with state-of-the-art FedAvg and VAE on UNSW-NB15 dataset.

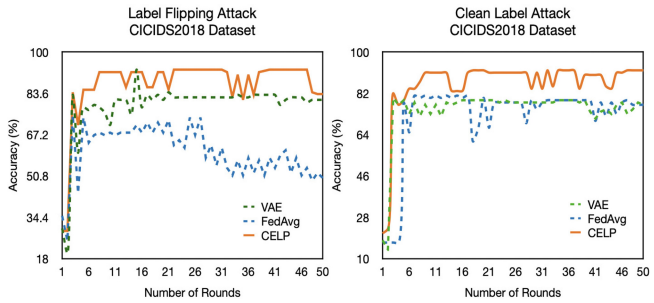

Fig. 12. Performance comparison of accuracy on multi-round attack scenario with state-of-the-art FedAvg and VAE on CICIDS2018 dataset.

Figure 12 depicts CELP's performance on the CICIDS2018 dataset, further illustrating its robustness against complex attack patterns. The fluctuation observed in accuracy, particularly in the presence of Non-IID data, highlights the challenge of model distribution shifts a common occurrence in real-world scenarios where data across clients is not identically distributed. CELP's design counters this instability through its eligibility-based client selection, which ensures that a broader and more representative set of client updates are aggregated, thus enhancing the overall stability and accuracy of the

model. The comparison with FedAvg and VAE in this figure underscores CELP's superior defense mechanism, enabled by the continuous monitoring capabilities of the integrated TA. Such monitoring is crucial for recognizing and neutralizing the subtle and evolving strategies employed in poisoning attacks.

These results collectively affirm CELP's advanced capabilities in maintaining model integrity and learning accuracy in the face of sophisticated multi-round attacks, setting a new benchmark for security in FL systems.

IX. Comparative Analysis

In this section, we provide a detailed comparison of CELP with existing solutions that are also designed for resource-constrained environments. The primary focus is on demonstrating how CELP not only meets but exceeds the capabilities of these alternatives through innovative features and efficient execution.

Comparison Metrics:

- *Model Efficiency:* CELP uses sample-based pruning to reduce model size effectively, which is crucial in environments with limited computational resources. We compare this with other methods that may not prioritize model size reduction, resulting in slower processing times and higher resource consumption.
- *Resource Allocation:* Unlike many FL protocols that assume uniform resource distribution among clients, CELP uniquely incorporates a CES that assesses and selects clients based on their actual resource availability, ensuring more efficient resource utilization.
- *System Adaptability:* CELP's re-parameterization of the FedAvg algorithm allows for partial task completion, which adapts to the heterogeneous capabilities of network clients more flexibly than traditional FL models.
- *Security Measures:* With an integrated IDS, CELP enhances security and trust in the system by monitoring and countering potential threats, a feature that is often overlooked in similar protocols.

Empirical Validation: We conducted several experiments to compare the performance of CELP with other leading solutions in this domain. The results, as illustrated in above Figures 6-12, show that CELP achieves superior convergence rates and maintains high accuracy levels even under stringent resource constraints. Detailed performance metrics and benchmarks are provided, highlighting areas where CELP provides significant improvements over alternatives such as FedAvg, FedProx, and VAE. This comparative analysis not only underscores the advantages of CELP in handling resource-constrained environments but also showcases its broader applicability and effectiveness in real-world FL scenarios.

X. Conclusion

This paper introduced the Client Eligibility-based Lightweight Protocol (CELP), a novel approach tailored for federated FL in resource-constrained environments. CELP effectively integrates a re-parameterized client objective function with a model pruning mechanism and resource-aware client selection, specifically designed to manage heterogeneous networks. Additionally, CELP accommodates partial task completion by resource-constrained clients, enabling robust convergence even under challenging conditions. Our simulation experiments demonstrate that CELP significantly enhances performance metrics: it achieves up to 93% accuracy on the MNIST dataset and 83% accuracy on CIFAR-10, markedly improving over existing approaches. Furthermore, CELP reduces communication costs by up to 81.01% compared to FedAvg and 72.54% compared to FedProx, effectively handling high proportions of straggler clients. The integration of a NIDS also plays a crucial role in enhancing security, effectively identifying and mitigating malicious activities, and contributing to the superior accuracy and performance of the system. These results underscore CELP's potential to transform FL applications by ensuring high efficiency and robust security in diverse operational scenarios.

References

[1] M. Asad, S. Otoum, and O. Al Fandi, "Edge computing for the metaverse: Balancing security and privacy concerns," in *Proc. Int. Conf. Intell. Metaverse Technol. Appl. (iMETA)*, 2023, pp. 1–8.

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[3] Z. Chen, D. Li, R. Ni, J. Zhu, and S. Zhang, "FedSeq: A hybrid federated learning framework based on sequential in-cluster training," *IEEE Syst. J.*, vol. 17, no. 3, pp. 4038–4049, Sep. 2023.

[4] A. Taik, A. Abouaomar, and S. Cherkaoui, "Green federated learning-based models and protocols," in *Green Machine Learning Protocols for Future Communication Networks*. Boca Raton, FL, USA: CRC Press, 2024, pp. 63–102.

[5] O. Shamir, N. Srebro, and T. Zhang, "Communication-efficient distributed optimization using an approximate Newton-type method," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 1000–1008.

[6] Z. Li and P. Richtárik, "A unified analysis of stochastic gradient methods for nonconvex federated optimization," 2020, *arXiv:2006.07013*.

[7] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, pp. 429–450.

[8] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "Federated learning for resource-constrained IoT devices: Panoramas and state-of-the-art," 2020, *arXiv:2002.10610*.

[9] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Proc. 25th Eur. Symp. Res. Comput. Secur. (ESORICS)*, Guildford, U.K., 2020, pp. 480–501.

[10] M. Asad and S. Otoum, "Integrative federated learning and zero-trust approach for secure wireless communications," *IEEE Wireless Commun.*, vol. 31, no. 2, pp. 14–20, Apr. 2024.

[11] S. Wang, F. Roosta, P. Xu, and M. W. Mahoney, "Giant: Globally improved approximate Newton method for distributed optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 1–11.

[12] Y. Arjevani and O. Shamir, "Communication complexity of distributed convex learning and optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1–9.

[13] C. Battiloro, P. Di Lorenzo, M. Merluzzi, and S. Barbarossa, "Lyapunov-based optimization of edge resources for energy-efficient adaptive federated learning," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 1, pp. 265–280, Mar. 2023.

[14] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.

[15] M. Asad, A. Moustafa, F. A. Rabhi, and M. Aslam, "THF: 3-way hierarchical framework for efficient client selection and resource management in federated learning," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11085–11097, Jul. 2022.

[16] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends® Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.

[17] O. Dekel, R. Gilad-Bachrach, O. Shamir, and L. Xiao, "Optimal distributed online prediction using mini-batches," *J. Mach. Learn. Res.*, vol. 13, no. 1, pp. 165–202, 2012.
[18] C. T. Dinh, N. Tran, and J. Nguyen, "Personalized federated learning with Moreau envelopes," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21394–21405.
[19] W. Huang, M. Ye, and B. Du, "Learn from others and be yourself in heterogeneous federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10143–10153.
[20] V. Smith, C.-K. Chiang, M. Sanjabi, and A. S. Talwalkar, "Federated multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.
[21] A. Khaled, K. Mishchenko, and P. Richtárik, "Better communication complexity for local SGD," 2019, Preprint.
[22] G. Malinovskiy, D. Kovalev, E. Gasanov, L. Condat, and P. Richtarik, "From local SGD to local fixed-point methods for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 6692–6701.
[23] B. Woodworth et al., "Is local SGD better than minibatch SGD?" in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 10334–10343.
[24] D. Li and W. Fedmd, "Heterogenous federated learning via model distillation," in *Proc. NeurIPS Workshop Feder. Learn. Data Privacy Confident.*, 2019, p. 8.
[25] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 269–283, Jan. 2021.
[26] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 1935–1949, Mar. 2021.
[27] F. Haddadpour and M. Mahdavi, "On the convergence of local descent methods in federated learning," 2019, *arXiv:1910.14425*.
[28] H. T. Nguyen, V. Sehwag, S. Hosseinalipour, C. G. Brinton, M. Chiang, and H. V. Poor, "Fast-convergent federated learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 201–218, Jan. 2021.
[29] H. Guo, A. Liu, and V. K. Lau, "Analog gradient aggregation for federated learning over wireless networks: Customized design and convergence analysis," *IEEE Internet Things J.*, vol. 8, no. 1, pp. 197–210, Jan. 2021.
[30] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-efficient on-device machine learning: Federated distillation and augmentation under non-IID private data," 2018, *arXiv:1811.11479*.
[31] L. Huang, Y. Yin, Z. Fu, S. Zhang, H. Deng, and D. Liu, "LoAdaBoost: Loss-based AdaBoost federated machine learning with reduced computational complexity on IID and non-IID intensive care data," *Plos one*, vol. 15, no. 4, 2020, Art. no. e0230706.
[32] M. Asad and S. Otoum, "Towards privacy-aware federated learning for user-sensitive data," in *Proc. 5th Int. Conf. Blockchain Comput. Appl. (BCCA)*, 2023, pp. 343–350.
[33] J. Dong, D. Zhang, Y. Cong, W. Cong, H. Ding, and D. Dai, "Federated incremental semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3934–3943.
[34] J. Dong et al., "Federated class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10164–10173.
[35] J. Le, D. Zhang, X. Lei, L. Jiao, K. Zeng, and X. Liao, "Privacy-preserving federated learning with malicious clients and honest-but-curious servers," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4329–4344, 2023.
[36] A. E. Cinà et al., "Wild patterns reloaded: A survey of machine learning security against training data poisoning," *ACM Comput. Surveys*, vol. 55, no. 13, pp. 1–39, 2023.
[37] A. K. Singh, A. Blanco-Justicia, and J. Domingo-Ferrer, "Fair detection of poisoning attacks in federated learning on non-IID data," *Data Min. Knowl. Discovery*, vol. 37, pp. 1998–2023, Sep. 2023.
[38] M. Anisetti, C. A. Ardagna, A. Balestrucci, N. Bena, E. Damiani, and C. Y. Yeun, "On the robustness of random forest against untargeted data poisoning: An ensemble-based approach," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 4, pp. 540–554, Dec. 2023.
[39] C. Fung, C. J. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," 2018, *arXiv:1808.04866*.
[40] M. A. Önsü, B. Kantarci, and A. Boukerche, "On the impact of malicious and cooperative clients on validation score-based model aggregation for federated learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2023, pp. 1634–1639.
[41] M. Asad et al., "Limitations and future aspects of communication costs in federated learning: A survey," *Sensors*, vol. 23, no. 17, p. 7358, 2023.
[42] M. Gecer and B. Garbinato, "Federated learning for mobility applications," *ACM Comput. Surveys*, vol. 56, no. 5, pp. 1–28, 2024.
[43] M. Asad, S. Shaukat, E. Javanmardi, J. Nakazato, N. Bao, and M. Tsukada, "Secure and efficient blockchain-based federated learning approach for VANETs," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 9047–9055, Mar. 2024.
[44] M. Aslam et al., "Adaptive machine learning based distributed denial-of-services attacks detection and mitigation system for SDN-enabled IoT," *Sensors*, vol. 22, no. 7, p. 2697, 2022.
[45] Z. Wang, J. Nakazato, M. Asad, E. Javanmardi, and M. Tsukada, "Overcoming environmental challenges in CAVs through MEC-based federated learning," in *Proc. 14th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, 2023, pp. 151–156.
[46] Y. Jiang et al., "Model pruning enables efficient federated learning on edge devices," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10374–10386, Dec. 2023.
[47] W. Xu, W. Fang, Y. Ding, M. Zou, and N. Xiong, "Accelerating federated learning for IoT in big data analytics with pruning, quantization and selective updating," *IEEE Access*, vol. 9, pp. 38457–38466, 2021.
[48] H. Zhang, J. Liu, J. Jia, Y. Zhou, H. Dai, and D. Dou, "FedDUAP: Federated learning with dynamic update and adaptive pruning using shared data on the server," 2022, *arXiv:2204.11536*.
[49] Ş. Cobzaş, R. Miculescu, and A. Nicolae, *Lipschitz Functions*, vol. 10. Cham, Switzerland: Springer, 2019.
[50] "TensorFlow federated." Accessed: Sep. 22, 2022. [Online]. Available: https://www.tensorflow.org/federated
[51] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.
[52] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSp*, 2018, pp. 108–116.
[53] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[54] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
[55] A. K. Sahu, T. Li, M. Sanjabi, M. Zaheer, A. Talwalkar, and V. Smith, "On the convergence of federated optimization in heterogeneous networks," 2018, *arXiv:1812.06127*.
[56] T. D. Nguyen, P. Rieger, M. Miettinen, and A.-R. Sadeghi, "Poisoning attacks on federated learning-based IoT intrusion detection system," in *Proc. Workshop Decentralized IoT Syst. Secur.(DISS)*, 2020, pp. 1–7.
[57] W. Liu et al., "D2MIF: A malicious model detection mechanism for federated-learning-empowered Artificial Intelligence of Things," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2141–2151, Feb. 2023.
[58] S. Li, Y. Cheng, W. Wang, Y. Liu, and T. Chen, "Learning to detect malicious clients for robust federated learning," 2020, *arXiv:2002.00211*.
[59] S. M. Kasongo and Y. Sun, "A deep learning method with wrapper based feature extraction for wireless intrusion detection system," *Comput. Secur.*, vol. 92, May 2020, Art. no. 101752.

Muhammad Asad (Member, IEEE) received the bachelor's degree in telecommunication and networking from COMSATS University Islamabad, Pakistan, the master's degree in computer science from the Dalian University of Technology, China, and the Ph.D. degree in computer science from the Nagoya Institute of Technology, Japan. He is currently working as a Postdoctoral Researcher with Zayed University, Abu Dhabi, UAE. He also works as a Visiting Researcher with the University of Tokyo, Japan. Prior to these roles, he worked as a Project Researcher with the Department of Creative Informatics, University of Tokyo. His research primarily focuses on federated learning, deep learning, and a range of advanced networking technologies, including VANETs, ITS, WSNs, SDN, and IoT, showcasing his deep expertise and ongoing contributions to the field of computer science. He received the prestigious President's Award from the Nagoya Institute of Technology in 2021–2022 for his outstanding research achievements.

Safa Otoum (Member, IEEE) received the M.A.Sc. and Ph.D. degrees in computer engineering from the University of Ottawa, Canada, in 2015 and 2019, respectively. She is an Assistant Professor of Computer Engineering with the College of Technological Innovation (CTI), Zayed University, UAE, and a Researcher in the field of communications and network security. Prior to joining CTI, she was a Postdoctoral Fellow with the University of Ottawa and has been a Data Scientist with Cheetah Networks Inc., Ottawa, since 2019. Her research interests include network security, blockchain applications, applications of ML and Aim, IoT, intrusion detection, and prevention systems. She received several academic and research scholarships, including the prestigious NSERC Canada Graduate Scholarships-Doctoral, the NSERC FSS, and the RIF-Zayed University Grant. She is actively working on several reputable events within IEEE and ACM. She is currently a Professional Engineer Ontario.

Saima Shaukat received the bachelor's degree in computer science from the Department of Computer Science, The University of Lahore, Pakistan, in 2016, and the master's degree in computer science from the Department of Computer Science, COMSATS University Islamabad (Lahore Campus), Pakistan, in September 2020. She is currently working as a Technical Researcher with the Graduate School of Information Science and Technology, The University of Tokyo, Japan. Previously, she worked as a Junior Lecturer with Superior University, Lahore, and a Lecturer with Lahore Garrison University. Her research interests include distributed machine learning, natural language processing, and Internet of Things.